

Project FORESIGHT

Phase 0 — Project Foundation, Reproducibility and Governance Validation

Project Type: Demand Forecasting and Inventory Intelligence

Notebook: 00_Phase_0_Setup.ipynb

Author: Mohit Kumar

Status: Completed

Project Overview

Project FORESIGHT is an end-to-end demand forecasting and inventory intelligence project designed to transform raw retail data into validated datasets, analytical insights, forecasting features, predictive models and decision-support applications.

This Phase 0 notebook establishes and validates the project foundation required by the downstream analytical workflow. It does not perform exploratory analysis, feature engineering or model training. Instead, it confirms that the project structure, development scope, reproducibility files, pipeline runner, documentation and governance controls are correctly established.

Purpose of This Notebook

This notebook performs four foundational responsibilities:

1. Defines and validates the fixed 300 SKU-store development scope.
2. Records the Python environment required for reproducibility.
3. Creates and verifies the automated Notebook 01 data-pipeline runner.
4. Audits the project structure, roadmap, decisions, assumptions and limitations.

Successful completion of these checks confirms that Project FORESIGHT has a documented, reproducible and governed foundation for its downstream data engineering, analysis, feature engineering and forecasting stages.

Important: This is a project-control and validation notebook. It is not an analytical or modelling notebook.

Position in the Project Workflow

Notebook 00 defines the governance and reproducibility foundation for the complete Project FORESIGHT workflow:

Stage	Primary Responsibility
Phase 0 — Setup	Project foundation, reproducibility and governance validation
Notebook 01	Data engineering and dataset validation
Notebook 02	Exploratory data analysis and business insights
Notebook 03	Weekly feature engineering and forecasting handoff
Notebook 04 onward	Baseline forecasting, advanced models, evaluation and decision-support delivery

Although this notebook is numbered `00` because its **conceptual role comes before the analytical workflow**, one of its validation tasks reads the processed Sales Daily dataset produced by Notebook 01. Therefore, in a completely fresh project setup, Notebook 01 must first produce `sales_daily_final.csv` before every Notebook 00 cell can execute successfully.

The `00` numbering represents the notebook's architectural position and governance purpose; it does not mean that every cell is independent of downstream-generated project files.

Prerequisites

Before executing this notebook, the following conditions must be satisfied:

- The notebook is located inside the `notebooks` directory.
- The Project FORESIGHT folder structure has been created.
- Notebook 01 has produced the validated file `data/processed/sales_daily_final.csv`.
- The project documentation files exist inside the `docs` directory.
- The `.gitignore` file exists in the project root.
- The required Python packages are installed in the active environment.

Primary Inputs

This notebook reads or validates the following project assets:

- `data/processed/sales_daily_final.csv`
- `docs/development_scope.md`
- `docs/project_roadmap.md`
- `docs/decision_log.md`
- `docs/assumptions_limitations.md`
- `.gitignore`
- Existing project folders and required deliverable files

Generated Outputs

The notebook creates or updates the following reproducibility and governance assets:

- `data/processed/development_sku_scope.csv`
- `requirements.txt`
- `run_data_pipeline.py`
- `reports/tables/project_roadmap_audit.csv`
- `reports/tables/project_roadmap_lock_audit.csv`
- `reports/tables/decision_log_audit.csv`
- `reports/tables/assumptions_limitations_audit.csv`
- `reports/tables/stage_1_governance_audit.csv`

Scope Boundary

This notebook does **not** clean raw data, perform exploratory analysis, engineer forecasting features, train predictive models or create dashboards. Those responsibilities belong to the downstream notebooks and delivery stages.

Its responsibility is limited to establishing reproducibility controls, documenting the fixed development scope and validating that the project foundation remains complete and internally consistent.

1. Development Scope Definition

1.1 Create the Development SKU-Store Scope

Project FORESIGHT uses a controlled development scope of **300 SKU-store time series**, created from:

- 30 category-balanced development products;
- 10 retail stores;
- all 1,913 available historical sales days.

Here, a `sku_id` identifies a specific **product-store combination**, not only an individual product. Therefore:

$30 \text{ products} \times 10 \text{ stores} = 300 \text{ SKU-store series}$

The following cell reads the validated `sales_daily_final.csv` dataset produced by Notebook 01 and extracts one unique record for every `sku_id`. These identifiers are sorted and exported as:

```
data/processed/development_sku_scope.csv
```

Why This File Is Required

The exported scope file acts as the canonical reference for the fixed 300-series development boundary. It makes the selected scope explicit, reviewable and reproducible without requiring a user to infer it from the complete daily sales dataset.

This is a **scope-control artifact**, not an additional analytical dataset. It records which SKU-store series belong to the defined local-development pipeline.

Validation Performed

Before export, the cell confirms that:

- exactly 300 SKU-store identifiers are present;
- every `sku_id` is unique;
- no `sku_id` is missing.

If any condition fails, execution stops through an assertion rather than exporting an invalid scope file.

Dependency note: This step requires `sales_daily_final.csv` from Notebook 01. It does not modify that source dataset.

```
In [1]: from pathlib import Path

import pandas as pd

# Locate the Project FORESIGHT folders.
PROJECT_ROOT = Path.cwd().parent
PROCESSED_DATA_DIR = PROJECT_ROOT / "data" / "processed"

# Read only the SKU identifier from the validated Sales Daily dataset.
sales_scope_source = pd.read_csv(
    PROCESSED_DATA_DIR / "sales_daily_final.csv",
    usecols=["sku_id"]
)

# Create one unique record for every SKU-store series in the development scope.
development_sku_scope = (
    sales_scope_source
    .drop_duplicates()
    .sort_values("sku_id")
    .reset_index(drop=True)
)

# Validate the development scope before export.
assert len(development_sku_scope) == 300, (
    "Expected exactly 300 development SKUs."
)
assert development_sku_scope["sku_id"].is_unique, (
    "Duplicate SKU identifiers were detected."
)
assert development_sku_scope["sku_id"].notna().all(), (
    "Missing SKU identifiers were detected."
)

# Export the validated scope file.
scope_file = (
    PROCESSED_DATA_DIR
    / "development_sku_scope.csv"
)
```

```
development_sku_scope.to_csv(
    scope_file,
    index=False
)

print("✅ Development scope file created successfully.")
print(f"File location : {scope_file}")
print(f"Total rows      : {len(development_sku_scope):,}")
print(
    "Unique SKUs   : "
    f"{development_sku_scope['sku_id'].nunique():,}"
)

display(development_sku_scope.head())
```

✅ Development scope file created successfully.
 File location : C:\Users\hp\Project FORESIGHT\data\processed\development_sku_scope.csv
 Total rows : 300
 Unique SKUs : 300

	sku_id
0	FOODS_1_001_CA_1
1	FOODS_1_001_CA_2
2	FOODS_1_001_CA_3
3	FOODS_1_001_CA_4
4	FOODS_1_001_TX_1

2. Environment Reproducibility

2.1 Generate the Pinned Requirements File

A reproducible project should document the software packages and exact versions used to execute its workflow. Without version information, changes between package releases may produce different behaviour, warnings or results on another computer.

The following cell identifies the versions currently installed for the packages required by the Notebook 01 data-engineering pipeline:

- `pandas` for tabular data processing;
- `numpy` for numerical operations;
- `notebook` for the Jupyter Notebook environment;
- `ipykernel` for executing Python within Jupyter;
- `nbconvert` for automated notebook execution and export.

Each successfully detected package is recorded using a pinned version format:

```
package_name==version_number
```

The resulting list is written to:

requirements.txt

in the Project FORESIGHT root directory.

Why Version Pinning Is Important

Pinned versions allow another user to recreate the same core environment instead of automatically installing potentially incompatible future releases. This improves:

- reproducibility;
- environment consistency;
- debugging reliability;
- project handover;
- portfolio review.

The cell also verifies that `requirements.txt` was created successfully and is not empty. If a required package is unavailable in the active environment, the cell displays a warning for that package.

Maintenance note: At this stage, `requirements.txt` records the packages required by the Notebook 01 data pipeline and its automated execution. Packages required for forecasting models, dashboards and deployment will be added when those project stages are finalized. Rerunning this cell overwrites the existing file with the package list defined in the cell.

```
In [2]: from importlib.metadata import PackageNotFoundError, version
        from pathlib import Path
```

```
# Locate the Project FORESIGHT root directory.
PROJECT_ROOT = Path.cwd().parent
```

```
# Packages currently required to run Notebook 01.
required_packages = [
```

```
    "pandas",
    "numpy",
    "notebook",
    "ipykernel",
    "nbconvert",
```

```
]
```

```
requirements = []
```

```
for package in required_packages:
```

```
    try:
```

```
        package_version = version(package)
        requirements.append(
            f"{package}=={package_version}"
        )
```

```
    except PackageNotFoundError:
```

```
        print(
            f"⚠ Package not found: {package}"
        )
```

```

# Define the output file.
requirements_file = (
    PROJECT_ROOT / "requirements.txt"
)

# Save the package list.
requirements_file.write_text(
    "\n".join(requirements) + "\n",
    encoding="utf-8"
)

# Validate the created file.
assert requirements_file.exists(), (
    "requirements.txt was not created."
)
assert requirements_file.stat().st_size > 0, (
    "requirements.txt is empty."
)

print("✅ requirements.txt created successfully.")
print(f"File location: {requirements_file}")
print("\nPackages recorded:")

for requirement in requirements:
    print(f"- {requirement}")

```

✅ requirements.txt created successfully.
File location: C:\Users\hp\Project FORESIGHT\requirements.txt

Packages recorded:

- pandas==3.0.3
- numpy==2.4.6
- notebook==7.5.7
- ipykernel==7.2.0
- nbconvert==7.17.1

2.2 Verify the Project Configuration Files

The project root contains two important configuration files:

- `.gitignore`, which prevents temporary, generated or environment-specific files from being unnecessarily tracked by Git;
- `requirements.txt`, which records the pinned Python package versions required for reproducible pipeline execution.

The following cell verifies that both files:

- exist in the Project FORESIGHT root directory;
- are valid files rather than missing paths;
- contain data and are not empty.

For transparency, the cell also displays the file locations and previews the first 12 lines of `.gitignore`. This allows the project's exclusion rules to be reviewed directly from the notebook.

If either file is missing or empty, an assertion stops execution and reports the configuration problem.

Validation boundary: This check confirms file availability and non-empty content. It does not test every `.gitignore` rule or reinstall the packages recorded in `requirements.txt`.

```
In [3]: from pathlib import Path

# Locate the project root.
PROJECT_ROOT = Path.cwd().parent

# Define the two project configuration files.
gitignore_file = PROJECT_ROOT / ".gitignore"
requirements_file = PROJECT_ROOT / "requirements.txt"

# Check whether both files exist.
print("Project configuration verification")
print("-" * 45)
print(f".gitignore exists      : {gitignore_file.exists()}")
print(f"requirements.txt exists : {requirements_file.exists()}")

# Validate both files.
assert gitignore_file.exists(), (
    ".gitignore was not found in the project root."
)

assert requirements_file.exists(), (
    "requirements.txt was not found in the project root."
)

assert gitignore_file.stat().st_size > 0, (
    ".gitignore is empty."
)

assert requirements_file.stat().st_size > 0, (
    "requirements.txt is empty."
)

print("\n✅ Both project configuration files exist and contain data.")

print("\n.gitignore location:")
print(gitignore_file)

print("\nrequirements.txt location:")
print(requirements_file)

print("\n.gitignore preview:")
print("-" * 45)

gitignore_lines = gitignore_file.read_text(
    encoding="utf-8"
).splitlines()

for line in gitignore_lines[:12]:
    print(line)
```

Project configuration verification

```
-----  
.gitignore exists      : True  
requirements.txt exists : True
```

✅ Both project configuration files exist and contain data.

.gitignore location:
C:\Users\hp\Project FORESIGHT\.gitignore

requirements.txt location:
C:\Users\hp\Project FORESIGHT\requirements.txt

.gitignore preview:

```
-----  
# -----  
# Python cache files  
# -----  
__pycache__/  
*.py[cod]  
  
# -----  
# Jupyter temporary files  
# -----  
.ipynb_checkpoints/  
  
# -----
```

3. Automated Data-Pipeline Runner

3.1 Create the Notebook 01 Pipeline Runner

A reproducible data pipeline should be executable through a consistent command instead of requiring a user to open Notebook 01 and run every cell manually.

The following cell creates:

```
run_data_pipeline.py
```

in the Project FORESIGHT root directory. This standalone Python script automates the complete **Notebook 01 data-engineering workflow**.

Automated Workflow

When executed, the runner:

1. Locates `01_Data_Engineering_Validation.ipynb` .
2. Re-executes Notebook 01 in place through Jupyter `nbconvert` .
3. Stops and reports an error if notebook execution fails.
4. Verifies the existence of the four primary processed datasets:
 - `sales_daily_final.csv`
 - `calendar_final.csv`
 - `sku_master_final.csv`
 - `inventory_snapshots_final.csv`

5. Recreates `development_sku_scope.csv` from the newly produced Sales Daily dataset.
6. Validates that the scope contains exactly 300 unique, non-missing SKU-store identifiers.
7. Confirms all five expected pipeline outputs and reports their file sizes.

From the Project FORESIGHT root directory, the generated runner can be executed with:

```
python run_data_pipeline.py
```

Why This Runner Is Required

The runner provides:

- repeatable execution of the data-engineering workflow;
- a single entry point for regenerating processed datasets;
- automatic failure reporting;
- output-existence verification;
- consistent recreation of the fixed development-scope file.

Automation Boundary

This script automates **Notebook 01 only**. It does not execute Notebook 00, exploratory analysis, feature engineering, forecasting models, dashboards or deployment components.

The four main dataset checks confirm that the expected files exist after Notebook 01 completes. Detailed schema, quality and cross-dataset validations remain the responsibility of Notebook 01 itself.

Execution caution: Running `run_data_pipeline.py` re-executes Notebook 01 and may overwrite its saved outputs and the five processed CSV files. It should therefore be used intentionally when the data-engineering pipeline needs to be regenerated.

Maintenance note: Rerunning the following Notebook 00 cell overwrites the existing `run_data_pipeline.py` file with the script version defined inside that cell.

```
In [4]: from pathlib import Path

PROJECT_ROOT = Path.cwd().parent
runner_file = PROJECT_ROOT / "run_data_pipeline.py"

runner_code = '''Execute the Project FORESIGHT data pipeline from one command

from pathlib import Path
import subprocess
import sys
```

```

import pandas as pd

PROJECT_ROOT = Path(__file__).resolve().parent
NOTEBOOK_DIR = PROJECT_ROOT / "notebooks"
NOTEBOOK_PATH = NOTEBOOK_DIR / "01_Data_Engineering_Validation.ipynb"
PROCESSED_DATA_DIR = PROJECT_ROOT / "data" / "processed"

MAIN_OUTPUTS = [
    "sales_daily_final.csv",
    "calendar_final.csv",
    "sku_master_final.csv",
    "inventory_snapshots_final.csv",
]

SCOPE_OUTPUT = "development_sku_scope.csv"

def create_development_scope() -> None:
    """Create the validated 300-SKU development-scope file."""

    sales_file = (
        PROCESSED_DATA_DIR
        / "sales_daily_final.csv"
    )

    sales_scope_source = pd.read_csv(
        sales_file,
        usecols=["sku_id"],
    )

    development_scope = (
        sales_scope_source
        .drop_duplicates()
        .sort_values("sku_id")
        .reset_index(drop=True)
    )

    assert len(development_scope) == 300, (
        "Expected exactly 300 development SKUs."
    )

    assert development_scope["sku_id"].is_unique, (
        "Duplicate SKU identifiers were detected."
    )

    assert development_scope["sku_id"].notna().all(), (
        "Missing SKU identifiers were detected."
    )

    scope_file = (
        PROCESSED_DATA_DIR
        / SCOPE_OUTPUT
    )

    development_scope.to_csv(
        scope_file,
        index=False,
    )

```

```

def main() -> None:
    """Execute Notebook 01 and verify all pipeline outputs."""

    print("=" * 70)
    print("PROJECT FORESIGHT – DATA PIPELINE")
    print("=" * 70)

    if not NOTEBOOK_PATH.exists():
        raise FileNotFoundError(
            f"Notebook not found: {NOTEBOOK_PATH}"
        )

    print(f"Notebook: {NOTEBOOK_PATH}")
    print("Starting clean notebook execution...")
    print("This process may take several minutes.\\n")

    command = [
        sys.executable,
        "-m",
        "jupyter",
        "nbconvert",
        "--to",
        "notebook",
        "--execute",
        "--inplace",
        "--ExecutePreprocessor.timeout=-1",
        NOTEBOOK_PATH.name,
    ]

    completed_process = subprocess.run(
        command,
        cwd=NOTEBOOK_DIR,
        check=False,
    )

    if completed_process.returncode != 0:
        raise RuntimeError(
            "Notebook execution failed. Review the error shown above."
        )

    missing_main_outputs = [
        file_name
        for file_name in MAIN_OUTPUTS
        if not (PROCESSED_DATA_DIR / file_name).exists()
    ]

    if missing_main_outputs:
        raise FileNotFoundError(
            "Expected outputs are missing: "
            f"{missing_main_outputs}"
        )

    create_development_scope()

    all_outputs = MAIN_OUTPUTS + [SCOPE_OUTPUT]

    missing_outputs = [
        file_name
        for file_name in all_outputs

```

```

        if not (PROCESSED_DATA_DIR / file_name).exists()
    ]

    if missing_outputs:
        raise FileNotFoundError(
            "Pipeline outputs are missing: "
            f"{missing_outputs}"
        )

    print("\n" + "=" * 70)
    print("✅ DATA PIPELINE COMPLETED SUCCESSFULLY")
    print("=" * 70)
    print("Verified outputs:")

    for file_name in all_outputs:
        output_path = PROCESSED_DATA_DIR / file_name
        file_size_mb = output_path.stat().st_size / (1024 ** 2)

        print(
            f"- {file_name} "
            f"({file_size_mb:.2f} MB)"
        )

if __name__ == "__main__":
    main()
...

runner_file.write_text(
    runner_code,
    encoding="utf-8",
)

assert runner_file.exists()
assert runner_file.stat().st_size > 0

print("✅ Pipeline runner updated successfully.")
print(f"File location: {runner_file}")
print(f"File size      : {runner_file.stat().st_size:,} bytes")
print("Automated outputs: 5")

```

✅ Pipeline runner updated successfully.
File location: C:\Users\hp\Project FORESIGHT\run_data_pipeline.py
File size : 3,668 bytes
Automated outputs: 5

4. Project Structure Validation

4.1 Audit the Required Project Folders and Files

A professional analytics project requires a predictable directory structure so that datasets, notebooks, reports, source code, models and delivery applications remain organized and discoverable.

The following cell performs a final Phase 0 audit of the minimum Project FORESIGHT structure. It validates:

- **25 required folders** covering data, documentation, notebooks, reports, reusable source code, models, testing and delivery applications;
- **16 required files** covering project configuration, governance, raw data, Notebook 01, its PDF export and the validated processed datasets.

Together, these produce:

25 folder checks + 16 file checks = 41 structural checks

Validation Logic

For every required folder, the audit confirms that the path:

- exists;
- is a directory.

For every required file, it confirms that the path:

- exists;
- is a file;
- contains data and is not empty.

The results are assembled into a validation table with the following fields:

Field	Meaning
Type	Whether the checked asset is a folder or file
Path	Expected location relative to the project root
Status	Passed , Missing or Missing or Empty

If any required asset fails validation, an assertion stops execution and lists the affected paths.

Why This Audit Is Required

This check protects the downstream workflow from structural failures such as:

- missing input datasets;
- misplaced notebooks;
- absent configuration files;
- missing reporting directories;
- empty required deliverables;
- incomplete project architecture.

It also gives reviewers a transparent view of the project components expected at the completion of Phase 0 and Notebook 01.

Validation Boundary

This cell **does not create, delete or modify** the folders and files being checked. It only validates their presence and basic file integrity.

A **Passed** result confirms structural availability, not the internal correctness of every file. Detailed dataset schema and quality validation remains the responsibility of Notebook 01, while later notebooks validate their own analytical and forecasting outputs.

Workflow note: This structural audit verifies the project foundation only. Formal readiness for Notebook 02 is confirmed later by the final Stage 1 Governance Audit.

```
In [5]: from pathlib import Path

import pandas as pd

# Locate the Project FORESIGHT root directory.
PROJECT_ROOT = Path.cwd().parent

# Required project folders.
required_folders = [
    "api",
    "dashboard",
    "data",
    "data/raw",
    "data/processed",
    "data/interim",
    "data/dashboard",
    "docs",
    "models",
    "notebooks",
    "presentation",
    "reports",
    "reports/executive_report",
    "reports/figures",
    "reports/notebook_exports",
    "reports/tables",
    "src",
    "src/data",
    "src/evaluation",
    "src/features",
    "src/inventory",
    "src/models",
    "src/utils",
    "streamlit_app",
    "tests",
]

# Required project files.
required_files = [
    "README.md",
    "requirements.txt",
    ".gitignore",
    "run_data_pipeline.py",
    "docs/development_scope.md",
    "notebooks/00_Phase_0_Setup.ipynb",
    "notebooks/01_Data_Engineering_Validation.ipynb",
    "reports/notebook_exports/01_Data_Engineering_Validation.pdf",
    "data/raw/calendar.csv",
    "data/raw/sales_train_validation.csv.gz",
```

```

        "data/raw/sell_prices.csv.gz",
        "data/processed/sales_daily_final.csv",
        "data/processed/calendar_final.csv",
        "data/processed/sku_master_final.csv",
        "data/processed/inventory_snapshots_final.csv",
        "data/processed/development_sku_scope.csv",
    ]

    audit_records = []

    # Check folders.
    for relative_path in required_folders:
        full_path = PROJECT_ROOT / relative_path
        passed = full_path.exists() and full_path.is_dir()

        audit_records.append({
            "Type": "Folder",
            "Path": relative_path,
            "Status": "Passed" if passed else "Missing",
        })

    # Check files.
    for relative_path in required_files:
        full_path = PROJECT_ROOT / relative_path
        passed = (
            full_path.exists()
            and full_path.is_file()
            and full_path.stat().st_size > 0
        )

        audit_records.append({
            "Type": "File",
            "Path": relative_path,
            "Status": "Passed" if passed else "Missing or Empty",
        })

    phase_0_audit = pd.DataFrame(audit_records)

    failed_items = phase_0_audit[
        phase_0_audit["Status"] != "Passed"
    ].copy()

    print("PROJECT FORESIGHT – PHASE 0 FINAL AUDIT")
    print("=" * 60)
    print(f"Total checks : {len(phase_0_audit)}")
    print(
        "Passed          : "
        f"{phase_0_audit['Status'].eq('Passed').sum()}"
    )
    print(f"Failed           : {len(failed_items)}")

    display(phase_0_audit)

    assert failed_items.empty, (
        "Phase 0 audit failed for:\n"
        + failed_items[["Type", "Path", "Status"]].to_string(
            index=False
        )
    )

```

```
print("\n✅ All Phase 0 folders and files passed validation.")
print("✅ Project foundation is ready for Stage 1 governance validation.")
```

PROJECT FORESIGHT – PHASE 0 FINAL AUDIT

=====

Total checks : 41

Passed : 41

Failed : 0

	Type	Path	Status
0	Folder	api	Passed
1	Folder	dashboard	Passed
2	Folder	data	Passed
3	Folder	data/raw	Passed
4	Folder	data/processed	Passed
5	Folder	data/interim	Passed
6	Folder	data/dashboard	Passed
7	Folder	docs	Passed
8	Folder	models	Passed
9	Folder	notebooks	Passed
10	Folder	presentation	Passed
11	Folder	reports	Passed
12	Folder	reports/executive_report	Passed
13	Folder	reports/figures	Passed
14	Folder	reports/notebook_exports	Passed
15	Folder	reports/tables	Passed
16	Folder	src	Passed
17	Folder	src/data	Passed
18	Folder	src/evaluation	Passed
19	Folder	src/features	Passed
20	Folder	src/inventory	Passed
21	Folder	src/models	Passed
22	Folder	src/utils	Passed
23	Folder	streamlit_app	Passed
24	Folder	tests	Passed
25	File	README.md	Passed
26	File	requirements.txt	Passed
27	File	.gitignore	Passed
28	File	run_data_pipeline.py	Passed
29	File	docs/development_scope.md	Passed
30	File	notebooks/00_Phase_0_Setup.ipynb	Passed
31	File	notebooks/01_Data_Engineering_Validation.ipynb	Passed
32	File	reports/notebook_exports/01_Data_Engineering_V...	Passed

	Type	Path	Status
33	File	data/raw/calendar.csv	Passed
34	File	data/raw/sales_train_validation.csv.gz	Passed
35	File	data/raw/sell_prices.csv.gz	Passed
36	File	data/processed/sales_daily_final.csv	Passed
37	File	data/processed/calendar_final.csv	Passed
38	File	data/processed/sku_master_final.csv	Passed
39	File	data/processed/inventory_snapshots_final.csv	Passed
40	File	data/processed/development_sku_scope.csv	Passed

- ✓ All Phase 0 folders and files passed validation.
- ✓ Project foundation is ready for Stage 1 governance validation.

5. Project Governance Validation

Project FORESIGHT uses formal governance documents to preserve the decisions, scope, assumptions and delivery commitments that guide the complete workflow.

The following audits validate that these documents exist, contain the required information and remain internally consistent with the project's locked design.

5.1 Validate the Project Roadmap Structure

The project roadmap is the central planning document for Project FORESIGHT. It defines the project stages, required deliverables, technical approach, acceptance criteria and progression from data engineering to forecasting and decision-support delivery.

The following cell validates:

```
docs/project_roadmap.md
```

Validation Performed

The structural audit confirms that:

- the roadmap file exists and is not empty;
- its required sections and project commitments are present;
- the roadmap status is `Locked` ;
- all 22 acceptance-checklist items are completed;
- no acceptance item remains pending;
- the document contains the required forecasting, evaluation, modelling and delivery references.

Each requirement is evaluated separately and recorded as either `Passed` or `Failed` .

The complete audit result is exported to:

```
reports/tables/project_roadmap_audit.csv
```

If any required condition fails, an assertion stops execution and identifies the failed roadmap checks.

Why This Audit Is Required

The roadmap acts as a controlled project contract. Validating it prevents the implementation from gradually moving away from the approved scope, methodology or deliverables without a documented decision.

A successful result confirms that the roadmap is structurally complete and reflects the current finalized project direction.

Relationship to the Next Audit

This structural audit checks the roadmap's overall content and completion state. The following **Roadmap Lock Audit** performs a more focused verification of the final locked decisions and acceptance criteria.

Current-state note: An earlier development-stage version of this audit expected 20 completed and 2 pending items while the roadmap was still `In Progress`. The project roadmap is now finalized, so the executable audit correctly expects `Locked` status, 22 completed items and zero pending items.

```
In [6]: from pathlib import Path
import re

import pandas as pd

# Locate the Project FORESIGHT root directory.
CURRENT_DIRECTORY = Path.cwd()

if CURRENT_DIRECTORY.name.lower() == "notebooks":
    PROJECT_ROOT = CURRENT_DIRECTORY.parent
else:
    PROJECT_ROOT = CURRENT_DIRECTORY

ROADMAP_PATH = PROJECT_ROOT / "docs" / "project_roadmap.md"
AUDIT_OUTPUT_PATH = (
    PROJECT_ROOT
    / "reports"
    / "tables"
    / "project_roadmap_audit.csv"
)

audit_records = []

def add_check(check_name, condition, details):
    """Add one validation result to the audit records."""
```

```

        audit_records.append({
            "Check": check_name,
            "Status": "Passed" if condition else "Failed",
            "Details": details,
        })

# -----
# 1. File-level checks
# -----

roadmap_exists = ROADMAP_PATH.exists() and ROADMAP_PATH.is_file()

add_check(
    "Roadmap file exists",
    roadmap_exists,
    str(ROADMAP_PATH),
)

if roadmap_exists:
    roadmap_text = ROADMAP_PATH.read_text(encoding="utf-8")
    roadmap_size = ROADMAP_PATH.stat().st_size
else:
    roadmap_text = ""
    roadmap_size = 0

add_check(
    "Roadmap file is not empty",
    roadmap_size > 5000,
    f"{roadmap_size:,} bytes",
)

add_check(
    "Correct roadmap filename",
    ROADMAP_PATH.name == "project_roadmap.md",
    ROADMAP_PATH.name,
)

# -----
# 2. Main section checks
# -----

required_sections = [
    f"## {section_number}."
    for section_number in range(1, 20)
]

missing_sections = [
    section
    for section in required_sections
    if section not in roadmap_text
]

add_check(
    "Sections 1 to 19 are present",
    len(missing_sections) == 0,
    (
        "All required sections found"
        if not missing_sections
    )
)

```

```

        else f"Missing: {missing_sections}"
    ),
)

# -----
# 3. Deliverable checks
# -----

required_deliverables = [
    "**D1**",
    "**D2**",
    "**D3**",
    "**D4**",
    "**D5**",
    "**D6**",
    "**D7**",
]

missing_deliverables = [
    deliverable
    for deliverable in required_deliverables
    if deliverable not in roadmap_text
]

add_check(
    "All seven client deliverables are present",
    len(missing_deliverables) == 0,
    (
        "D1 through D7 found"
        if not missing_deliverables
        else f"Missing: {missing_deliverables}"
    ),
)

# -----
# 4. Locked technical-decision checks
# -----

required_decisions = [
    "Weekly SKU-store level",
    "Eight weeks",
    "WAPE",
    "Forecast Bias",
    "Seasonal-naive forecast",
    "Rolling-origin cross-validation",
    "LightGBM",
    "XGBoost",
    "300 SKU-store series",
]

missing_decisions = [
    decision
    for decision in required_decisions
    if decision not in roadmap_text
]

add_check(
    "Locked forecasting decisions are present",

```

```

len(missing_decisions) == 0,
(
    "All locked decisions found"
    if not missing_decisions
    else f"Missing: {missing_decisions}"
),
)

# -----
# 5. Project-adaptation checks
# -----

required_disclosures = [
    "sales_train_validation.csv.gz",
    "calendar.csv",
    "sell_prices.csv.gz",
    "inventory_snapshots_final.csv",
    "simulated",
    "development_sku_scope.csv",
]

missing_disclosures = [
    disclosure
    for disclosure in required_disclosures
    if disclosure not in roadmap_text
]

add_check(
    "M5 adaptation and simulation disclosures are present",
    len(missing_disclosures) == 0,
    (
        "All disclosures found"
        if not missing_disclosures
        else f"Missing: {missing_disclosures}"
    ),
)

# -----
# 6. Roadmap acceptance-checklist checks
# -----

acceptance_match = re.search(
    r"## 18\. Roadmap Acceptance Checklist(?:.*)"
    r"## 19\. Roadmap Lock Declaration",
    roadmap_text,
    flags=re.DOTALL,
)

if acceptance_match:
    acceptance_text = acceptance_match.group(1)
    completed_items = acceptance_text.count("- [x]")
    pending_items = acceptance_text.count("- [ ]")
else:
    completed_items = 0
    pending_items = 0

add_check(
    "All 22 acceptance items are completed",

```

```

        completed_items == 22,
        f"Completed items found: {completed_items}",
    )

    add_check(
        "No acceptance items remain pending",
        pending_items == 0,
        f"Pending items found: {pending_items}",
    )

# -----
# 7. Document-control checks
# -----

    add_check(
        "Roadmap version is 3.0",
        "| **Roadmap Version** | 3.0 |" in roadmap_text,
        "Expected version: 3.0",
    )

    add_check(
        "Roadmap status is Locked",
        "| **Status** | Locked |" in roadmap_text,
        "Expected current status: Locked",
    )

# -----
# 8. Markdown formatting check
# -----

    code_fence_count = roadmap_text.count("`")

    add_check(
        "Markdown code fences are balanced",
        code_fence_count % 2 == 0,
        f"Code-fence markers found: {code_fence_count}",
    )

# -----
# 9. Export and final validation
# -----

    roadmap_audit = pd.DataFrame(audit_records)

    AUDIT_OUTPUT_PATH.parent.mkdir(
        parents=True,
        exist_ok=True,
    )

    roadmap_audit.to_csv(
        AUDIT_OUTPUT_PATH,
        index=False,
    )

    failed_checks = roadmap_audit[
        roadmap_audit["Status"] == "Failed"
    ].copy()

```

```

print("PROJECT FORESIGHT – ROADMAP STRUCTURAL AUDIT")
print("=" * 60)
print(f"Roadmap size : {roadmap_size:,} bytes")
print(f"Total checks : {len(roadmap_audit)}")
print(
    "Passed      : "
    f"{roadmap_audit['Status'].eq('Passed').sum()}"
)
print(f"Failed      : {len(failed_checks)}")

display(roadmap_audit)

assert failed_checks.empty, (
    "Roadmap audit failed:\n"
    + failed_checks.to_string(index=False)
)

print("\n✅ Roadmap structural audit passed.")
print(f"✅ Audit exported to: {AUDIT_OUTPUT_PATH}")
print("✅ The locked roadmap passed structural validation.")

```

PROJECT FORESIGHT – ROADMAP STRUCTURAL AUDIT

```

=====
Roadmap size : 42,329 bytes
Total checks : 12
Passed      : 12
Failed      : 0

```

	Check	Status	Details
0	Roadmap file exists	Passed	C:\Users\hp\Project FORESIGHT\docs\project_roa...
1	Roadmap file is not empty	Passed	42,329 bytes
2	Correct roadmap filename	Passed	project_roadmap.md
3	Sections 1 to 19 are present	Passed	All required sections found
4	All seven client deliverables are present	Passed	D1 through D7 found
5	Locked forecasting decisions are present	Passed	All locked decisions found
6	M5 adaptation and simulation disclosures are p...	Passed	All disclosures found
7	All 22 acceptance items are completed	Passed	Completed items found: 22
8	No acceptance items remain pending	Passed	Pending items found: 0
9	Roadmap version is 3.0	Passed	Expected version: 3.0
10	Roadmap status is Locked	Passed	Expected current status: Locked
11	Markdown code fences are balanced	Passed	Code-fence markers found: 2

- ✓ Roadmap structural audit passed.
- ✓ Audit exported to: C:\Users\hp\Project FORESIGHT\reports\tables\project_roadmap_audit.csv
- ✓ The locked roadmap passed structural validation.

5.2 Validate the Locked Roadmap State

After the overall roadmap structure has been validated, this audit confirms that the finalized roadmap remains in its formally approved and locked state.

The following cell performs 10 focused checks on:

```
docs/project_roadmap.md
```

Validation Performed

The audit confirms that:

- the roadmap file exists;
- the roadmap version is 3.0 ;
- the document status is Locked ;
- Stage 1 — Roadmap and Project Governance is marked Completed ;
- Stage 13 — Final Documentation remains In Progress ;
- all 22 roadmap acceptance items are completed;
- no acceptance item remains pending;
- roadmap Sections 1 through 19 remain present;
- the final lock declaration is included;
- Markdown code fences remain balanced.

The results are exported to:

```
reports/tables/project_roadmap_lock_audit.csv
```

If any requirement fails, an assertion stops execution and displays the failed checks.

Why Stage 13 Remains In Progress

A locked roadmap does not mean that the complete Project FORESIGHT implementation is already finished. It means that the approved project scope, architecture, methodology and deliverables have been finalized.

Stage 13 remains In Progress because documentation must continue throughout the remaining notebooks, modelling, evaluation and delivery stages.

Why This Lock Audit Is Required

The roadmap lock establishes a stable reference point for downstream work. It prevents silent changes to major commitments such as:

- the project scope and analytical architecture;
- the forecasting level and horizon;

- the validation methodology;
- the selected model families;
- the evaluation metrics;
- the required dashboards, API and reporting deliverables.

Any future change to these locked decisions should be intentional, documented in the decision log and reflected consistently across the relevant governance files.

Relationship to the Structural Audit

The previous audit verifies the roadmap's broader content and structural completeness. This audit specifically verifies its **final document-control state, stage statuses, completed acceptance checklist and lock integrity**.

A successful result confirms that Roadmap Version 3.0 remains formally locked and ready to govern the downstream Project FORESIGHT workflow.

```
In [7]: from pathlib import Path
import re

import pandas as pd

# Locate the project root.
CURRENT_DIRECTORY = Path.cwd()

if CURRENT_DIRECTORY.name.lower() == "notebooks":
    PROJECT_ROOT = CURRENT_DIRECTORY.parent
else:
    PROJECT_ROOT = CURRENT_DIRECTORY

ROADMAP_PATH = PROJECT_ROOT / "docs" / "project_roadmap.md"

LOCK_AUDIT_PATH = (
    PROJECT_ROOT
    / "reports"
    / "tables"
    / "project_roadmap_lock_audit.csv"
)

roadmap_file_available = (
    ROADMAP_PATH.exists()
    and ROADMAP_PATH.is_file()
)

roadmap_text = (
    ROADMAP_PATH.read_text(encoding="utf-8")
    if roadmap_file_available
    else ""
)

audit_records = []

def add_check(check_name, condition, details):
    """Store one roadmap-lock validation result."""
```

```

        audit_records.append({
            "Check": check_name,
            "Status": "Passed" if condition else "Failed",
            "Details": details,
        })

# -----
# 1. Document-control checks
# -----

add_check(
    "Roadmap file exists",
    roadmap_file_available,
    str(ROADMAP_PATH),
)

add_check(
    "Roadmap version is 3.0",
    "| **Roadmap Version** | 3.0 |" in roadmap_text,
    "Expected roadmap version: 3.0",
)

add_check(
    "Document status is Locked",
    "| **Status** | Locked |" in roadmap_text,
    "Expected document status: Locked",
)

# -----
# 2. Stage-status checks
# -----

stage_1_completed = re.search(
    r"### Stage 1 – Roadmap and Project Governance"
    r".*?\.*Status:\.* Completed",
    roadmap_text,
    flags=re.DOTALL,
)

add_check(
    "Stage 1 status is Completed",
    stage_1_completed is not None,
    "Roadmap and governance stage must be completed",
)

stage_13_in_progress = re.search(
    r"### Stage 13 – Final Documentation"
    r".*?\.*Status:\.* In Progress",
    roadmap_text,
    flags=re.DOTALL,
)

add_check(
    "Stage 13 remains In Progress",
    stage_13_in_progress is not None,
    "Documentation must continue throughout the project",
)

```

```

# -----
# 3. Acceptance-checklist checks
# -----

acceptance_match = re.search(
    r"## 18\. Roadmap Acceptance Checklist(.*)"
    r"## 19\. Roadmap Lock Declaration",
    roadmap_text,
    flags=re.DOTALL,
)

if acceptance_match:
    acceptance_text = acceptance_match.group(1)
    completed_items = acceptance_text.count("- [x]")
    pending_items = acceptance_text.count("- [ ]")
else:
    completed_items = 0
    pending_items = 0

add_check(
    "All 22 acceptance items are completed",
    completed_items == 22,
    f"Completed items found: {completed_items}",
)

add_check(
    "No acceptance items remain pending",
    pending_items == 0,
    f"Pending items found: {pending_items}",
)

# -----
# 4. Final integrity checks
# -----

required_sections = [
    f"## {section_number}."
    for section_number in range(1, 20)
]

missing_sections = [
    section
    for section in required_sections
    if section not in roadmap_text
]

add_check(
    "Sections 1 to 19 remain present",
    len(missing_sections) == 0,
    (
        "All required sections found"
        if not missing_sections
        else f"Missing: {missing_sections}"
    ),
)

add_check(
    "Lock declaration is present",

```

```

        "Status: Locked" in roadmap_text,
        "Section 19 must contain the final lock declaration",
    )

code_fence_count = roadmap_text.count("` ` ` `")

add_check(
    "Markdown code fences remain balanced",
    code_fence_count % 2 == 0,
    f"Code-fence markers found: {code_fence_count}",
)

# -----
# 5. Export and final validation
# -----

lock_audit = pd.DataFrame(audit_records)

LOCK_AUDIT_PATH.parent.mkdir(
    parents=True,
    exist_ok=True,
)

lock_audit.to_csv(
    LOCK_AUDIT_PATH,
    index=False,
)

failed_checks = lock_audit[
    lock_audit["Status"] == "Failed"
].copy()

print("PROJECT FORESIGHT – ROADMAP LOCK AUDIT")
print("=" * 60)
print(f"Total checks : {len(lock_audit)}")
print(
    "Passed          : "
    f"{lock_audit['Status'].eq('Passed').sum()}"
)
print(f"Failed          : {len(failed_checks)}")

display(lock_audit)

assert failed_checks.empty, (
    "Roadmap lock audit failed:\n"
    + failed_checks.to_string(index=False)
)

print("\n✅ Roadmap lock audit passed.")
print("✅ Project roadmap version 3.0 is formally locked.")
print(f"✅ Audit exported to: {LOCK_AUDIT_PATH}")

```

PROJECT FORESIGHT – ROADMAP LOCK AUDIT

```

=====
Total checks : 10
Passed       : 10
Failed       : 0

```

	Check	Status	Details
0	Roadmap file exists	Passed	C:\Users\hp\Project FORESIGHT\docs\project_roa...
1	Roadmap version is 3.0	Passed	Expected roadmap version: 3.0
2	Document status is Locked	Passed	Expected document status: Locked
3	Stage 1 status is Completed	Passed	Roadmap and governance stage must be completed
4	Stage 13 remains In Progress	Passed	Documentation must continue throughout the pro...
5	All 22 acceptance items are completed	Passed	Completed items found: 22
6	No acceptance items remain pending	Passed	Pending items found: 0
7	Sections 1 to 19 remain present	Passed	All required sections found
8	Lock declaration is present	Passed	Section 19 must contain the final lock declara...
9	Markdown code fences remain balanced	Passed	Code-fence markers found: 2

- ✓ Roadmap lock audit passed.
- ✓ Project roadmap version 3.0 is formally locked.
- ✓ Audit exported to: C:\Users\hp\Project FORESIGHT\reports\tables\project_roadmap_lock_audit.csv

5.3 Validate the Project Decision Log

The decision log preserves the major technical, analytical and delivery choices made during Project FORESIGHT. It records not only what was selected, but also the reasoning, constraints and consequences associated with those decisions.

The following cell validates:

```
docs/decision_log.md
```

Validation Performed

The audit confirms that the decision log:

- exists and is not empty;
- contains all 22 documented project decisions;
- preserves the required decision sections and status information;
- records the approved forecasting scope, methodology, models, metrics and delivery commitments;
- remains consistent with the locked project roadmap.

The validated decisions include key commitments such as:

- weekly SKU-store forecasting;

- an eight-week forecast horizon;
- a controlled 300 SKU-store development scope;
- rolling-origin time-series validation;
- a seasonal-naive baseline;
- WAPE and Forecast Bias as primary evaluation metrics;
- LightGBM and XGBoost as advanced model families;
- explicit disclosure of simulated inventory data;
- Streamlit, Power BI and FastAPI delivery components.

Each required decision is evaluated individually and recorded as either **Passed** or **Failed**.

The complete validation table is exported to:

```
reports/tables/decision_log_audit.csv
```

If any required decision is missing or inconsistent, an assertion stops execution and identifies the failed checks.

Why the Decision Log Is Required

A roadmap defines what the project intends to deliver, while the decision log records the important choices made while designing that delivery.

This provides:

- traceability for technical decisions;
- protection against undocumented scope changes;
- consistent implementation across notebooks and applications;
- clear justification for modelling and evaluation choices;
- transparency for reviewers and future maintainers.

Governance Boundary

This audit confirms that the required decisions are documented. It does not independently prove that every decision has already been implemented in code.

Implementation evidence is produced progressively by the relevant notebooks, models, dashboards, API and final project documentation.

If a locked project decision changes in the future, the decision log, roadmap and related validation rules must be updated together so that the governance documents remain consistent.

```
In [8]: from pathlib import Path
import re

import pandas as pd

# Locate the Project FORESIGHT root folder.
```

```

CURRENT_DIRECTORY = Path.cwd()

if CURRENT_DIRECTORY.name.lower() == "notebooks":
    PROJECT_ROOT = CURRENT_DIRECTORY.parent
else:
    PROJECT_ROOT = CURRENT_DIRECTORY

DECISION_LOG_PATH = PROJECT_ROOT / "docs" / "decision_log.md"

AUDIT_OUTPUT_PATH = (
    PROJECT_ROOT
    / "reports"
    / "tables"
    / "decision_log_audit.csv"
)

audit_records = []

def add_check(check_name, condition, details):
    """Store one decision-log audit result."""
    audit_records.append({
        "Check": check_name,
        "Status": "Passed" if condition else "Failed",
        "Details": details,
    })

# -----
# 1. Read the decision log
# -----

file_exists = (
    DECISION_LOG_PATH.exists()
    and DECISION_LOG_PATH.is_file()
)

if file_exists:
    decision_text = DECISION_LOG_PATH.read_text(
        encoding="utf-8"
    )
    file_size = DECISION_LOG_PATH.stat().st_size
else:
    decision_text = ""
    file_size = 0

# -----
# 2. Basic file checks
# -----

add_check(
    "Decision log exists",
    file_exists,
    str(DECISION_LOG_PATH),
)

add_check(
    "Decision log is not empty",
    file_size > 10000,

```

```

        f"{file_size:,} bytes",
    )

    add_check(
        "Correct filename",
        DECISION_LOG_PATH.name == "decision_log.md",
        DECISION_LOG_PATH.name,
    )

# -----
# 3. Document-control checks
# -----

    add_check(
        "Document version is 1.0",
        "| **Document Version** | 1.0 |" in decision_text,
        "Expected version: 1.0",
    )

    add_check(
        "Document status is Active",
        "| **Status** | Active |" in decision_text,
        "Expected status: Active",
    )

# -----
# 4. Required section checks
# -----

    required_sections = [
        "# Project FORESIGHT – Decision Log",
        "## 1. Purpose",
        "## 2. Decision Status Values",
        "## 3. Decision Entry Structure",
        "## 4. Project Decisions",
        "## 5. Future Decisions",
    ]

    missing_sections = [
        section
        for section in required_sections
        if section not in decision_text
    ]

    add_check(
        "Required sections are present",
        len(missing_sections) == 0,
        (
            "All required sections found"
            if not missing_sections
            else f"Missing: {missing_sections}"
        ),
    )

# -----
# 5. Decision identifier checks
# -----

```

```

expected_decisions = [
    f"DEC-{{number:03d}}"
    for number in range(1, 23)
]

missing_decisions = [
    decision
    for decision in expected_decisions
    if decision not in decision_text
]

add_check(
    "DEC-001 through DEC-022 are present",
    len(missing_decisions) == 0,
    (
        "All 22 decision identifiers found"
        if not missing_decisions
        else f"Missing: {missing_decisions}"
    ),
)

decision_headings = re.findall(
    r"^### (DEC-\d{3})\b",
    decision_text,
    flags=re.MULTILINE,
)

add_check(
    "Exactly 22 decision headings exist",
    len(decision_headings) == 22,
    f"Decision headings found: {len(decision_headings)}",
)

add_check(
    "Decision identifiers are unique",
    len(decision_headings) == len(set(decision_headings)),
    f"Unique identifiers found: {len(set(decision_headings))}",
)

# -----
# 6. Core topic checks
# -----

required_topics = [
    "M5 Data",
    "Simulate Inventory Snapshot Data",
    "300 SKU-Store Series",
    "Seven-Notebook Analytical Architecture",
    "Weekly SKU-Store Level",
    "Eight-Week Forecast Horizon",
    "WAPE",
    "Rolling-Origin Cross-Validation",
    "Seasonal-Naive Baseline",
    "LightGBM and XGBoost",
    "Transparent Rule-Based Inventory Risk Scoring",
    "Streamlit",
    "FastAPI",
    "Power BI",

```

```

        "One-Command Reproducible Pipeline",
        "Master Roadmap Version 3.0",
    ]

    missing_topics = [
        topic
        for topic in required_topics
        if topic not in decision_text
    ]

    add_check(
        "Core project decisions are documented",
        len(missing_topics) == 0,
        (
            "All core decision topics found"
            if not missing_topics
            else f"Missing: {missing_topics}"
        ),
    )

# -----
# 7. Markdown-formatting check
# -----

code_fence_count = decision_text.count("`")

add_check(
    "Markdown code fences are balanced",
    code_fence_count % 2 == 0,
    f"Code-fence markers found: {code_fence_count}",
)

# -----
# 8. Export and validate the audit
# -----

decision_log_audit = pd.DataFrame(audit_records)

AUDIT_OUTPUT_PATH.parent.mkdir(
    parents=True,
    exist_ok=True,
)

decision_log_audit.to_csv(
    AUDIT_OUTPUT_PATH,
    index=False,
)

failed_checks = decision_log_audit[
    decision_log_audit["Status"] == "Failed"
].copy()

print("PROJECT FORESIGHT – DECISION LOG AUDIT")
print("=" * 60)
print(f"File size      : {file_size:,} bytes")
print(f"Total checks   : {len(decision_log_audit)}")
print(
    "Passed        : "

```

```

    f"{decision_log_audit['Status'].eq('Passed').sum()}"
)
print(f"Failed      : {len(failed_checks)}")

display(decision_log_audit)

assert failed_checks.empty, (
    "Decision log audit failed:\n"
    + failed_checks.to_string(index=False)
)

print("\n✅ Decision log audit passed.")
print("✅ DEC-001 through DEC-022 are documented.")
print(f"✅ Audit exported to: {AUDIT_OUTPUT_PATH}")

```

PROJECT FORESIGHT – DECISION LOG AUDIT

=====

File size : 23,765 bytes
Total checks : 11
Passed : 11
Failed : 0

	Check	Status	Details
0	Decision log exists	Passed	C:\Users\hp\Project FORESIGHT\docs\decision_lo...
1	Decision log is not empty	Passed	23,765 bytes
2	Correct filename	Passed	decision_log.md
3	Document version is 1.0	Passed	Expected version: 1.0
4	Document status is Active	Passed	Expected status: Active
5	Required sections are present	Passed	All required sections found
6	DEC-001 through DEC-022 are present	Passed	All 22 decision identifiers found
7	Exactly 22 decision headings exist	Passed	Decision headings found: 22
8	Decision identifiers are unique	Passed	Unique identifiers found: 22
9	Core project decisions are documented	Passed	All core decision topics found
10	Markdown code fences are balanced	Passed	Code-fence markers found: 0

✅ Decision log audit passed.
 ✅ DEC-001 through DEC-022 are documented.
 ✅ Audit exported to: C:\Users\hp\Project FORESIGHT\reports\tables\decision_log_audit.csv

5.4 Validate the Assumptions and Limitations Register

Every forecasting and inventory-intelligence project depends on assumptions arising from its data, development environment, modelling approach and delivery constraints. These assumptions must be disclosed clearly so that analytical outputs are interpreted responsibly.

The following cell validates:

```
docs/assumptions_limitations.md
```

Validation Performed

The audit confirms that the document:

- exists under the correct filename and is not empty;
- uses Document Version `1.0` ;
- has the current status `Active` ;
- contains all 12 required sections;
- preserves the required project assumptions and disclosures;
- contains balanced Markdown code fences.

The required disclosures include:

- the three selected M5 source files;
- the simulated nature of the Inventory Snapshot dataset;
- the controlled 300 SKU-store development scope;
- the seasonal-naive forecasting baseline;
- rolling-origin time-series validation;
- the eight-week forecasting horizon;
- the rule that correlation must not be described as causation;
- the interpretation of risk outputs as decision-support indicators.

Each requirement is evaluated separately and recorded as either `Passed` or `Failed` .

The complete audit result is exported to:

```
reports/tables/assumptions_limitations_audit.csv
```

If any required condition fails, an assertion stops execution and identifies the affected checks.

Why the Document Remains Active

Unlike the locked roadmap, the assumptions and limitations register remains `Active` because it must evolve as the project progresses.

Later forecasting, model-evaluation, risk-scoring, dashboard, API and deployment stages may introduce additional assumptions or reveal new limitations. These must be added transparently without silently changing the locked project scope.

Why This Audit Is Required

This audit ensures that reviewers can distinguish between:

- observed source data and simulated information;
- analytical evidence and interpretation;

- forecasting results and operational decisions;
- project capabilities and known limitations.

Documenting these boundaries prevents misleading claims and improves the transparency, credibility and responsible use of Project FORESIGHT outputs.

Validation Boundary

This audit verifies the structure and presence of required disclosures. It does not independently prove that every assumption is correct or that every future limitation has already been identified.

The register must continue to be updated whenever a new material assumption, limitation or interpretation rule emerges during downstream project stages.

Maintenance note: Rerunning the audit overwrites `assumptions_limitations_audit.csv` with the current validation results.

```
In [9]: from pathlib import Path

import pandas as pd

# Locate the Project FORESIGHT root directory.
CURRENT_DIRECTORY = Path.cwd()

if CURRENT_DIRECTORY.name.lower() == "notebooks":
    PROJECT_ROOT = CURRENT_DIRECTORY.parent
else:
    PROJECT_ROOT = CURRENT_DIRECTORY

ASSUMPTIONS_PATH = (
    PROJECT_ROOT
    / "docs"
    / "assumptions_limitations.md"
)

AUDIT_OUTPUT_PATH = (
    PROJECT_ROOT
    / "reports"
    / "tables"
    / "assumptions_limitations_audit.csv"
)

audit_records = []

def add_check(check_name, condition, details):
    """Store one audit result."""
    audit_records.append({
        "Check": check_name,
        "Status": "Passed" if condition else "Failed",
        "Details": details,
    })
```

```

# -----
# 1. Read the document
# -----

file_exists = (
    ASSUMPTIONS_PATH.exists()
    and ASSUMPTIONS_PATH.is_file()
)

if file_exists:
    document_text = ASSUMPTIONS_PATH.read_text(
        encoding="utf-8"
    )
    file_size = ASSUMPTIONS_PATH.stat().st_size
else:
    document_text = ""
    file_size = 0

# -----
# 2. Basic file checks
# -----

add_check(
    "Assumptions file exists",
    file_exists,
    str(ASSUMPTIONS_PATH),
)

add_check(
    "Assumptions file is not empty",
    file_size > 10000,
    f"{file_size:,} bytes",
)

add_check(
    "Correct filename",
    ASSUMPTIONS_PATH.name == "assumptions_limitations.md",
    ASSUMPTIONS_PATH.name,
)

# -----
# 3. Document-control checks
# -----

add_check(
    "Correct document title",
    (
        "# Project FORESIGHT – Assumptions and Limitations"
        in document_text
    ),
    "Expected main title found",
)

add_check(
    "Document version is 1.0",
    "| **Document Version** | 1.0 |" in document_text,

```

```

        "Expected version: 1.0",
    )

    add_check(
        "Document status is Active",
        "| **Status** | Active |" in document_text,
        "Expected status: Active",
    )

# -----
# 4. Required section checks
# -----

required_sections = [
    "## 1. Purpose",
    "## 2. Project Adaptation",
    "## 3. Data Assumptions",
    "## 4. Simulated Inventory Assumptions",
    "## 5. Development-Scope Assumptions",
    "## 6. Financial Assumptions",
    "## 7. Exploratory Analysis Limitations",
    "## 8. Forecasting Assumptions and Limitations",
    "## 9. Risk-Scoring Limitations",
    "## 10. Dashboard and API Limitations",
    "## 11. Interpretation Rules",
    "## 12. Update Policy",
]

missing_sections = [
    section
    for section in required_sections
    if section not in document_text
]

add_check(
    "Sections 1 through 12 are present",
    len(missing_sections) == 0,
    (
        "All required sections found"
        if not missing_sections
        else f"Missing: {missing_sections}"
    ),
)

# -----
# 5. Core disclosure checks
# -----

required_disclosures = [
    "sales_train_validation.csv.gz",
    "calendar.csv",
    "sell_prices.csv.gz",
    "Inventory Snapshot dataset is simulated",
    "300 SKU-store series",
    "seasonal-naive baseline",
    "rolling-origin",
    "eight weeks",
    "correlation must not be described as causation",

```

```

        "decision-support indicators",
    ]

    missing_disclosures = [
        disclosure
        for disclosure in required_disclosures
        if disclosure.lower() not in document_text.lower()
    ]

    add_check(
        "Core assumptions and disclosures are present",
        len(missing_disclosures) == 0,
        (
            "All core disclosures found"
            if not missing_disclosures
            else f"Missing: {missing_disclosures}"
        ),
    )

# -----
# 6. Markdown integrity check
# -----

code_fence_count = document_text.count("`")

add_check(
    "Markdown code fences are balanced",
    code_fence_count % 2 == 0,
    f"Code-fence markers found: {code_fence_count}",
)

# -----
# 7. Export and validate
# -----

assumptions_audit = pd.DataFrame(audit_records)

AUDIT_OUTPUT_PATH.parent.mkdir(
    parents=True,
    exist_ok=True,
)

assumptions_audit.to_csv(
    AUDIT_OUTPUT_PATH,
    index=False,
)

failed_checks = assumptions_audit[
    assumptions_audit["Status"] == "Failed"
].copy()

print("PROJECT FORESIGHT – ASSUMPTIONS DOCUMENT AUDIT")
print("=" * 60)
print(f"File size      : {file_size:,} bytes")
print(f"Total checks   : {len(assumptions_audit)}")
print(
    "Passed          : "
    f"{assumptions_audit['Status'].eq('Passed').sum()}"
)

```

```

)
print(f"Failed      : {len(failed_checks)}")

display(assumptions_audit)

assert failed_checks.empty, (
    "Assumptions document audit failed:\n"
    + failed_checks.to_string(index=False)
)

print("\n✅ Assumptions and limitations audit passed.")
print("✅ Core project disclosures are documented.")
print(f"✅ Audit exported to: {AUDIT_OUTPUT_PATH}")

```

PROJECT FORESIGHT – ASSUMPTIONS DOCUMENT AUDIT

=====

File size : 11,804 bytes
 Total checks : 9
 Passed : 9
 Failed : 0

	Check	Status	Details
0	Assumptions file exists	Passed	C:\Users\hp\Project FORESIGHT\docs\assumptions...
1	Assumptions file is not empty	Passed	11,804 bytes
2	Correct filename	Passed	assumptions_limitations.md
3	Correct document title	Passed	Expected main title found
4	Document version is 1.0	Passed	Expected version: 1.0
5	Document status is Active	Passed	Expected status: Active
6	Sections 1 through 12 are present	Passed	All required sections found
7	Core assumptions and disclosures are present	Passed	All core disclosures found
8	Markdown code fences are balanced	Passed	Code-fence markers found: 0

✅ Assumptions and limitations audit passed.
 ✅ Core project disclosures are documented.
 ✅ Audit exported to: C:\Users\hp\Project FORESIGHT\reports\tables\assumptions_1imitations_audit.csv

5.5 Perform the Final Stage 1 Governance Audit

The previous governance audits validate the roadmap, decision log and assumptions register individually. This final audit combines their essential requirements into a consolidated readiness check for **Stage 1 — Roadmap and Project Governance**.

Required Governance Assets

The cell verifies the availability and non-empty status of eight governance files:

1. docs/development_scope.md

2. docs/project_roadmap.md
3. docs/decision_log.md
4. docs/assumptions_limitations.md
5. reports/tables/project_roadmap_audit.csv
6. reports/tables/project_roadmap_lock_audit.csv
7. reports/tables/decision_log_audit.csv
8. reports/tables/assumptions_limitations_audit.csv

Cross-Document Validation

After confirming the required files, the audit performs six consolidated governance checks:

- Roadmap Version 3.0 is marked **Locked** ;
- Stage 1 — Roadmap and Project Governance is marked **Completed** ;
- all decisions from **DEC-001** through **DEC-022** are documented;
- the assumptions and limitations register remains **Active** ;
- the M5 source-data adaptation and simulated inventory are disclosed;
- **02_Exploratory_Data_Analysis.ipynb** is registered in the project roadmap.

Together, the cell performs:

8 required-file checks + 6 governance-content checks = 14 final checks

Each result is recorded as either **Passed** or **Failed** .

The consolidated audit table is exported to:

reports/tables/stage_1_governance_audit.csv

If any required file or governance condition fails, execution stops and reports the affected checks.

Why This Final Audit Is Required

This cell acts as the formal Stage 1 completion gate. It confirms that:

- the approved project scope is documented;
- the roadmap remains locked;
- major project decisions are traceable;
- assumptions and limitations are actively maintained;
- detailed governance-audit evidence exists;
- the next analytical notebook is registered in the approved workflow.

Interpretation of a Successful Result

A successful result confirms that the **project-governance foundation** is complete and that the workflow is authorized to proceed to Notebook 02.

It does not mean that the complete Project FORESIGHT implementation is finished. Feature engineering, forecasting, evaluation, inventory-risk analysis, dashboards, API development and final documentation remain downstream responsibilities.

Validation Boundary

For the four detailed audit CSV files, this final cell confirms only that they exist and are non-empty. Their individual validation logic and pass/fail results are produced by the preceding detailed audit cells.

Therefore, this notebook should be executed sequentially from top to bottom so that the detailed audit files are regenerated before the final Stage 1 audit runs.

Maintenance note: Rerunning this cell overwrites `stage_1_governance_audit.csv` with the latest consolidated results.

```
In [10]: from pathlib import Path

import pandas as pd

# Locate the Project FORESIGHT root directory.
CURRENT_DIRECTORY = Path.cwd()

if CURRENT_DIRECTORY.name.lower() == "notebooks":
    PROJECT_ROOT = CURRENT_DIRECTORY.parent
else:
    PROJECT_ROOT = CURRENT_DIRECTORY

# -----
# Required governance files
# -----

required_files = {
    "Development scope document":
        PROJECT_ROOT / "docs" / "development_scope.md",

    "Locked project roadmap":
        PROJECT_ROOT / "docs" / "project_roadmap.md",

    "Decision log":
        PROJECT_ROOT / "docs" / "decision_log.md",

    "Assumptions and limitations":
        PROJECT_ROOT / "docs" / "assumptions_limitations.md",

    "Roadmap structural audit":
        PROJECT_ROOT
        / "reports"
        / "tables"
        / "project_roadmap_audit.csv",

    "Roadmap lock audit":
        PROJECT_ROOT
        / "reports"
```

```

        / "tables"
        / "project_roadmap_lock_audit.csv",

    "Decision log audit":
        PROJECT_ROOT
        / "reports"
        / "tables"
        / "decision_log_audit.csv",

    "Assumptions audit":
        PROJECT_ROOT
        / "reports"
        / "tables"
        / "assumptions_limitations_audit.csv",
}

audit_records = []

def add_check(check_name, condition, details):
    """Store one Stage 1 audit result."""
    audit_records.append({
        "Check": check_name,
        "Status": "Passed" if condition else "Failed",
        "Details": details,
    })

# -----
# 1. Check required files
# -----

for file_name, file_path in required_files.items():
    valid_file = (
        file_path.exists()
        and file_path.is_file()
        and file_path.stat().st_size > 0
    )

    add_check(
        file_name,
        valid_file,
        str(file_path),
    )

# -----
# 2. Read governance documents
# -----

roadmap_path = required_files["Locked project roadmap"]
decision_log_path = required_files["Decision log"]
assumptions_path = required_files[
    "Assumptions and limitations"
]

roadmap_file_available = (
    roadmap_path.exists()
    and roadmap_path.is_file()

```

```

)

decision_log_file_available = (
    decision_log_path.exists()
    and decision_log_path.is_file()
)

assumptions_file_available = (
    assumptions_path.exists()
    and assumptions_path.is_file()
)

roadmap_text = (
    roadmap_path.read_text(encoding="utf-8")
    if roadmap_file_available
    else ""
)

decision_log_text = (
    decision_log_path.read_text(encoding="utf-8")
    if decision_log_file_available
    else ""
)

assumptions_text = (
    assumptions_path.read_text(encoding="utf-8")
    if assumptions_file_available
    else ""
)

# -----
# 3. Governance-content checks
# -----

add_check(
    "Roadmap version 3.0 is Locked",
    (
        "| **Roadmap Version** | 3.0 |"
        in roadmap_text
        and "| **Status** | Locked |"
        in roadmap_text
    ),
    "Expected locked roadmap version: 3.0",
)

add_check(
    "Stage 1 is marked Completed",
    (
        "### Stage 1 – Roadmap and Project Governance"
        in roadmap_text
        and "**Status:** Completed"
        in roadmap_text
    ),
    "Roadmap and governance stage",
)

add_check(
    "All 22 decisions are documented",
    all(

```

```

        f"DEC-{{number:03d}}" in decision_text
        for number in range(1, 23)
    ),
    "Expected decisions: DEC-001 through DEC-022",
)

add_check(
    "Assumptions document is Active",
    "| **Status** | Active |" in assumptions_text,
    "Assumptions will continue to be updated",
)

add_check(
    "M5 adaptation is disclosed",
    (
        "sales_train_validation.csv.gz"
        in assumptions_text
        and "inventory" in assumptions_text.lower()
        and "simulated" in assumptions_text.lower()
    ),
    "M5 source and simulated inventory disclosure",
)

add_check(
    "Notebook 02 is registered",
    (
        "02_Exploratory_Data_Analysis.ipynb"
        in roadmap_text
    ),
    "Notebook 02 must appear in the roadmap",
)

# -----
# 4. Export and validate
# -----

stage_1_audit = pd.DataFrame(audit_records)

AUDIT_OUTPUT_PATH = (
    PROJECT_ROOT
    / "reports"
    / "tables"
    / "stage_1_governance_audit.csv"
)

stage_1_audit.to_csv(
    AUDIT_OUTPUT_PATH,
    index=False,
)

failed_checks = stage_1_audit[
    stage_1_audit["Status"] == "Failed"
].copy()

print("PROJECT FORESIGHT – STAGE 1 FINAL AUDIT")
print("=" * 60)
print(f"Total checks : {len(stage_1_audit)}")
print(
    "Passed          : "

```

```

    f"{stage_1_audit['Status'].eq('Passed').sum()}"
)
print(f"Failed      : {len(failed_checks)}")

display(stage_1_audit)

assert failed_checks.empty, (
    "Stage 1 final audit failed:\n"
    + failed_checks.to_string(index=False)
)

print("\n✅ Stage 1 final audit passed.")
print("✅ Project governance documentation is complete.")
print("✅ Project FORESIGHT is ready for Notebook 02.")
print(f"✅ Audit exported to: {AUDIT_OUTPUT_PATH}")

```

PROJECT FORESIGHT – STAGE 1 FINAL AUDIT

=====

Total checks : 14

Passed : 14

Failed : 0

	Check	Status	Details
0	Development scope document	Passed	C:\Users\hp\Project FORESIGHT\docs\development...
1	Locked project roadmap	Passed	C:\Users\hp\Project FORESIGHT\docs\project_roa...
2	Decision log	Passed	C:\Users\hp\Project FORESIGHT\docs\decision_lo...
3	Assumptions and limitations	Passed	C:\Users\hp\Project FORESIGHT\docs\assumptions...
4	Roadmap structural audit	Passed	C:\Users\hp\Project FORESIGHT\reports\tables\p...
5	Roadmap lock audit	Passed	C:\Users\hp\Project FORESIGHT\reports\tables\p...
6	Decision log audit	Passed	C:\Users\hp\Project FORESIGHT\reports\tables\d...
7	Assumptions audit	Passed	C:\Users\hp\Project FORESIGHT\reports\tables\a...
8	Roadmap version 3.0 is Locked	Passed	Expected locked roadmap version: 3.0
9	Stage 1 is marked Completed	Passed	Roadmap and governance stage
10	All 22 decisions are documented	Passed	Expected decisions: DEC-001 through DEC-022
11	Assumptions document is Active	Passed	Assumptions will continue to be updated
12	M5 adaptation is disclosed	Passed	M5 source and simulated inventory disclosure
13	Notebook 02 is registered	Passed	Notebook 02 must appear in the roadmap

- ✅ Stage 1 final audit passed.
- ✅ Project governance documentation is complete.
- ✅ Project FORESIGHT is ready for Notebook 02.
- ✅ Audit exported to: C:\Users\hp\Project FORESIGHT\reports\tables\stage_1_governance_audit.csv

6. Notebook Completion Summary and Handoff

Notebook 00 establishes the reproducibility, structure and governance foundation for Project FORESIGHT.

Completed Responsibilities

This notebook:

- defines and exports the fixed 300 SKU-store development scope;
- generates the pinned project requirements file;
- verifies the core project configuration files;
- creates the automated Notebook 01 data-pipeline runner;
- validates the required project folders and files;
- audits the project roadmap and its locked state;
- validates the project decision log;
- validates the assumptions and limitations register;
- performs the consolidated Stage 1 governance audit.

Reproducibility Assets

The notebook creates or updates:

- `data/processed/development_sku_scope.csv`
- `requirements.txt`
- `run_data_pipeline.py`

Governance Evidence

The notebook produces:

- `reports/tables/project_roadmap_audit.csv`
- `reports/tables/project_roadmap_lock_audit.csv`
- `reports/tables/decision_log_audit.csv`
- `reports/tables/assumptions_limitations_audit.csv`
- `reports/tables/stage_1_governance_audit.csv`

Recommended Execution Order

For a completely fresh project environment:

1. Confirm that the three required raw M5 source files exist.
2. Execute Notebook 01 to generate the validated processed datasets.
3. Execute this Notebook 00 sequentially from top to bottom.
4. Confirm that every assertion and governance audit passes.
5. Proceed to Notebook 02 for exploratory data analysis.

For future pipeline regeneration, execute the following command from the Project FORESIGHT root directory:

```
python run_data_pipeline.py
```

After the pipeline completes, rerun Notebook 00 whenever the project foundation or governance evidence must be revalidated.

Successful Completion Criteria

Notebook 00 is successfully validated when:

- the development scope contains exactly 300 unique, non-missing SKU-store identifiers;
- `requirements.txt` and `.gitignore` exist and are not empty;
- `run_data_pipeline.py` is created successfully;
- all 41 project-structure checks pass;
- all detailed governance audits pass;
- all 14 final Stage 1 governance checks pass;
- no assertion or execution error remains.

Downstream Handoff

A successful execution confirms that the Phase 0 and Stage 1 foundation is ready to support:

```
02_Exploratory_Data_Analysis.ipynb
```

Notebook 02 is responsible for analysing historical demand, pricing, product performance, store performance, seasonality and inventory conditions. Its validated findings provide the analytical foundation for weekly feature engineering and forecasting in Notebook 03.

Final Status

Phase 0 — Project Foundation: Completed

Stage 1 — Roadmap and Project Governance: Completed

Next Workflow Stage: Notebook 02 — Exploratory Data Analysis

A successful notebook run confirms foundation and governance readiness. It does not represent completion of the full Project FORESIGHT forecasting and decision-support system.